Description-Similarity Rules: Towards Flexible Feature Engineering for Entity Matching

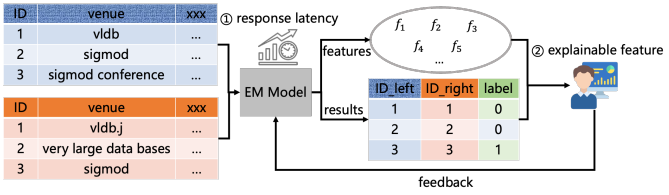
Yafeng Tang[1], Zheng Liang[1], Hongzhi Wang[1,✉], Xiaoou Ding[1], Tianyu Mu[1], Huan Hu[2]
[1] School of Computer Science, Harbin Institute of Technology, Harbin, China
[2] Huawei Cloud Computing Technologies Co., Ltd, Hangzhou, China
tangyf@stu.hit.edu.cn,{lz20, wangzh, dingxiaoou,mutianyu}@hit.edu.cn, huhuan18@huawei.com

***Abstract*—Entity Matching (EM) is a crucial task in data integration. Compared to deep learning-based EM solutions, tree-based machine learning models are more computationally effective and explainable, making them more applicable in real-world EM scenarios. However, Random Forest-based EM methods select features with a static feature engineering rule set for all attributes. Consequently, they suffer model retraining cost to select features, and can hardly customize to different EM tasks. To tackle this problem, we propose Description-Similarity Rules (DSR) for EM feature engineering. DSR introduces diverse attribute value distribution metrics and data-driven thresholds to traditional EM feature engineering rules. Unfortunately, both the DSR search space and its online model retraining costs are exponential. By pushing the model retraining to the offline stage, our DSR set mining algorithm is one order of magnitude faster than the baseline algorithms, taking only seconds for online selection. Empirically, DSR outperforms the feature engineering methods by 3.75% on average F1 score, while reaching the state-of-the-art EM performance on several datasets.**

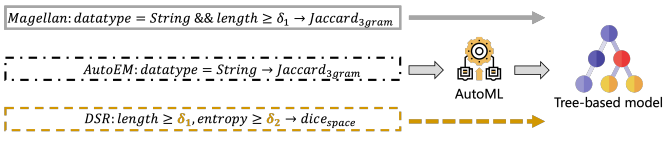
***Index Terms*—Entity Matching, Feature Engineering, Rule Mining**

I. Introduction

Entity matching [1]–[3] is the problem of identifying co-referent manifestations that refer to the same real-world entity from different data sources. For example, attributes of the same person from multiple sources are often complicated [4] in the real world. EM can improve data quality by reducing such duplicates among tables [5].



(a) An interactive EM pipeline



(b) Rule-based EM model

Fig. 1: A typical EM pipeline.

✉ Hongzhi Wang is the corresponding author.

EM has been an open problem for decades. Conventional EM solutions are based on similarity functions, which quantify the similarity between pairwise attribute values of an entity [6]. Rule-based [7]–[10] EM methods use certain thresholds for attribute similarities to make matching decisions, where experts specify the thresholds (e.g., the value of δ_1 in Figure 1(b)). Machine learning-based methods [6] train machine learning models on pairs of labeled entities with similarities as features. They search for the feature combination with the best evaluation metric (e.g., F1 score). Deep Learning methods [1], [3], [11]–[14] incorporate domain knowledge in language-model-based encoders. However, real-world EM tasks pose flexibility requirements beyond the grasp of these methods, which can be specified in the following aspects [15]. (1) **Response latency.** Entity Matching is rarely a one-time job. As Figure 1(a) shows, a typical EM pipeline requires data scientists to go through the cycle of exploring data [10], changing EM the solution, and checking results several times [16]. Also, EM becomes even more time-intensive on multi-source data which need to be split and matched separately. Methods with low response latency are necessary for refining and validating EM solutions in a pipeline as Figure 1. (2) **Explainable features.** To understand and reconfigure a machine learning model in the EM pipeline, one needs to understand how the model is influenced by its input, i.e., the feature value. Even for explainable EM models like Random Forest, system-selected features still need to be traceable from data [17].

To the best of our knowledge, few EM solutions can address the above two issues simultaneously. Rule-based methods like Autofuzzyjoin [18] may not perform well with less structured datasets and long textual data since they rely on limited word level similarity metrics and TF-IDF scores [19]. State-of-the-art deep learning methods like Sudowoodo [13] make it difficult for users to understand why entities are matched or unmatched. They often require a high cost for debugging the model: try numerous configurations at each iteration, and need expert experience to make decisions. Mainstream machine learning-based methods are easier to understand, but their efficiency is limited by searching models and features. Therefore, to meet the flexibility demands mentioned above, lightweight machine learning models such as Random Forest (RF) [20] have been generally adopted for EM [21] in commercial data integration pipelines [6], [15], [22] for their good explainability and satisfactory performance.

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00008

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00008

Example 1. *Consider a training process of a Random Forest-based EM model, a data scientist needs to construct features (i.e. similarities) for each attribute pair.*

In Figure 1(b), Magellan [23] simply uses length and data types to recommend similarities, so the selected features result in poor performance (Section V). AutoML-EM [6] relaxes the rule set of Magellan and recommends redundant features, which are subsequently trimmed by auto-sklearn [24]. The general feature selection methods [8], [25]–[29] search for the optimal feature at the cost of exhaustive model retraining. Considering efficiency, in this paper, we focus on rule-based feature selection methods.

Challenges. From Example 1, we can see that the bottleneck of the RF-based EM pipeline is the drawback of rule-based feature selection, which can be summarized as follows.

Exhaustive Rule Candidate Space. In a typical EM pipeline, rule-based similarity selection methods typically cost extensive computational resources as expert knowledge is unavailable. The threshold of descriptions [1] can take any value within a continuous range. Actually, we need to consider rules with conjunction of multiple descriptions as antecedents, and all similarities can be consequent. As a result, the rule candidate space is overwhelming, and searching for each rule candidate requires an EM model retraining. What's worse, an EM pipeline mostly lacks labeled data, so the hyperparameter search algorithms [30] fail to solve this problem.

Rule set is too rigid. The rule set of existing solutions only considers `length` and `datatype` of attribute pair, while the threshold is fixed for all EM tasks. They fail to offer users enough flexibility to give feedback and perform an EM task at hand. Consequently, such rigid rule sets with fixed thresholds recommend too few or insufficient features for complex datasets (as shown in Example 1). Users can barely understand EM data and give constructive feedback to the pipeline in such cases.

Description-Similarity Rule. Rule-based FE methods fall short in practice due to the above limitations. To unlock the flexibility of EM models, we need to design a flexible substitute of feature engineering rules without too much computational overhead. Motivated by this, we propose the Description-Similarity Rule (DSR). DSR helps users in EM pipeline to better understand data with the newly introduced "description". Also, DSR has a flexible threshold, which can be discovered from data without expert knowledge.

Contribution. The contributions of our work can be summarized as follows.

• We propose the Description-Similarity Rule (DSR), a set of data-driven rules for flexible Feature Engineering on Entity Matching models. We design *descriptions* to support common data types including sets, vectors, strings, and boolean data.

• To discover the optimal DSR set for a given EM task, we design a DSR mining algorithm accelerating the baseline approach by one order of magnitude (Section IV). Also, we address the problem of exponential online retraining by sampling historical datasets with a theoretical guarantee on error probability(Section III).

• Empirically, DSR beats the state-of-the-art EM solution by 3.75% on average F1 score. DSR is also statistically comparable to black-box Deep Learning methods (Section V).

[1]Description is a set metric to compute certain attribute value distributions. We will go into details in Definition 2

TABLE I: Frequently used notations.

Symbol	Meaning
T_1, T_2	A pair of tables for entity matching
$A_1, A_2, \ldots$	The attribute set of tables
t_1, t_2	An entity pair from T_1 and T_2
M	The entity matching model
p	The evaluation metric for the EM task
$D = \{d_1, d_2, d_3, d_4\}$	A set of descriptions
D'	A subset of descriptions
$S = \{S_1, S_2, \ldots\}$	A set of similarities
$R = \{r_1, r_2, \ldots\}$	A set of description-similarity rules
δ_i	The threshold for i-th description
θ	The threshold for lowest acceptable performance
ϕ	An operator
$dom(d_i)$	The domain of valid value of description
E	The expectation of a variable
n	The budget of EM model retraining times

II. Overview

In this section, we define essential concepts for Description-Similarity Rules (Section II-A), elaborate the design details of *Descriptions* (Section II-B), present the DSR recommendation problem(Section II-C), and give an overview of our solution (Section II-D).

A. Background

Entity Matching solutions for structured data consist of at least two phases: blocking and matching [31], and often assume that upstream tasks have prepared datasets with aligned attributes [6], [32]. In this paper, we focus on the feature engineering of entity matching for relational data and treat entity blocking as an orthogonal problem.

Definition 1. ***Entity Matching (EM):*** *Given two relational tables T_1 and T_2 with the aligned attribute set $A_1, A_2, ...$ with the same schema, Entity Matching determines whether each pair of records $t_1, t_2 \in T_1 \times T_2$ refers to the same entity in the real world.*

In this paper, we focus on the Random Forest(RF)-based EM solver, as RF is a widely-adopted machine learning model in real-world EM applications [6], [12], [22], [33]. For example, consider the entity pair with $ID = 3$ in Figure 1(a), "sigmod conference" and "sigmod" have a Jaccard similarity of 0.5, which is used by RF-based EM solvers as one of the features to represent this entity pair for RF training. Therefore, the core of *RF-based EM* is **similarity selection** [7], [34], [35], and we follow SOTA RF-based EM methods [6], [23] to exploit similarity selection rules.

A similarity function quantifies the similarity between two strings (or numeric or Boolean values). There are four types of similarity functions: set-based, sequence-based, vector-based,

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

and number-based. Next, we will introduce how we design *descriptions* to support the recommendation of these functions.

B. Preliminaries

Our intuition is that general FE methods [36], [37] or intuitive EM methods [6], [23] ignore latent attribute value statistics during similarity recommendation, thus cannot obtain ideal performance on EM task (see Section V for details). Therefore, we propose a new concept called *description* to overcome their drawbacks. For each *description*, we show their relationship with similarity functions.

Definition 2. ***Description (D):*** *Let $T_1.A_i$ and $T_2.A_i$ denote the set of the attribute value A_i of all entities in T_1 and T_2, respectively, the description $d \in D, D = \{d_1, d_2, d_3, d_4\}$ on attribute $d(A_i)$ can be computed as follows:*[2]

- `length`*: For sequence-based similarities such as Jaccard and Jaro [36], we present* `length` *as the length of the string in attribute A.*
- `AvgWordNum(AWN)`*: For set-based similarities, we present* `AWN` *to describe strings including separators.* `AWN` *is associated with similarity functions such as the dice-coefficient [38] and Jaccard Index [39].*

$d_2(AvgWordNum)$: Suppose N is a tokenizer. $N(A)$ denotes the tokenized wordBag after employing N on attribute A,

$$d_2(A_i) = |N(A_i)|$$

A tokenizer can be a publicly available tool [40], or any common delimiter such as spaces, commas, semicolons, etc.

- `Entropy`*: For vector-based similarities [41], we design* `entropy` *to convert strings into document vectors [42], especially for long texts.*

$d_3(entropy)$: For each attribute A, we treat A as a document, and $v(A_i)$ represents the vocabulary of A_i. The `entropy` *of A is calculated as follows:*

$$d_3(A_i) = -\sum_{j=1}^{|v(A_i)|} p_{ij} \cdot \log_2(p_{ij}) \tag{1}$$

where p_{ij} is the probability of the j-th word occurring in document A_i.

- `Index of Coincidence(IoC)`*: For number-based similarities, we present* `IoC` *[43] to identify the distribution of characters,* ***especially where numerical and textual values are mixed****.*

$d_4(IoC)$: We denote $v(A_i)$ as the vocabulary of A_i, and f_c as the frequency of character c in A_i. The `IoC` *of A_i is as follows:*

$$d_4(A_i) = \sum_{c \in v(A_i)} \frac{f_c(f_c - 1)}{|A_i|(|A_i| - 1)}$$

We explain the design of descriptions with Example 2. In our technical report [45], we also provide a formal basis to show that descriptions are actually building blocks of the equation of all four types of similarities.

[2]notations in Definition 2 will not be presented in Table I.

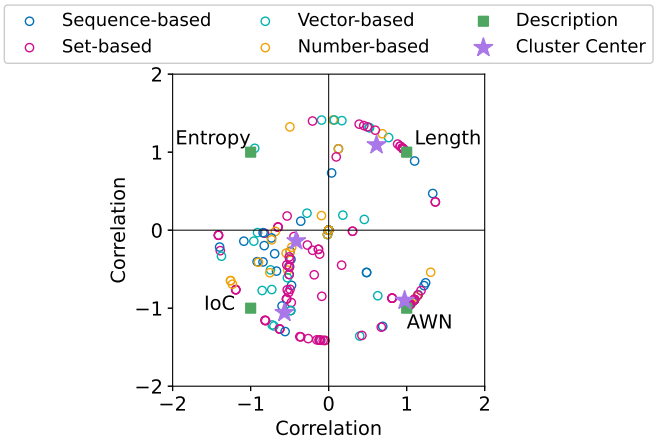


Fig. 2: The correlation diagram between four types of similarity functions (circles) and our descriptions (squares) on the Fodors-Zagats dataset [23] (better viewed in color). The purple stars represent the cluster centers of similarity functions, derived from the K-Means algorithm [44].

Example 2. *In Figure 2, we visualize the correlation between similarities and descriptions on the Fodors-Zagats dataset [23].First, we fix our four descriptions (square) in the center of each quadrant. Then, for each similarity, we calculate its Pearson correlation coefficient with each of the four descriptions and find a point (circle) whose Euclidean distance to each description is proportional to the corresponding coefficient. Note that every quadrant contains quite a few scattered circles. In addition, we apply K-Means clustering for the circles (similarities) and mark the center of the cluster with stars. Another interesting discovery is that three out of four stars land somewhere near a square. That means each description "attracts" considerable amount of similarities.*

As shown in Figure 2, descriptions can serve as the effective precomputation for all similarities. This empirical observation motivates us to explore the idea of recommending similarities based on four descriptions for a given EM task.

Under these definitions, we formally introduce DSR.

Definition 3. ***Description-Similarity Rule:*** *Given a Description set D and a Similarity Function set $S = \{S_1, S_2, \ldots\}$, the Description-Similarity-Rule (DSR) r is:*

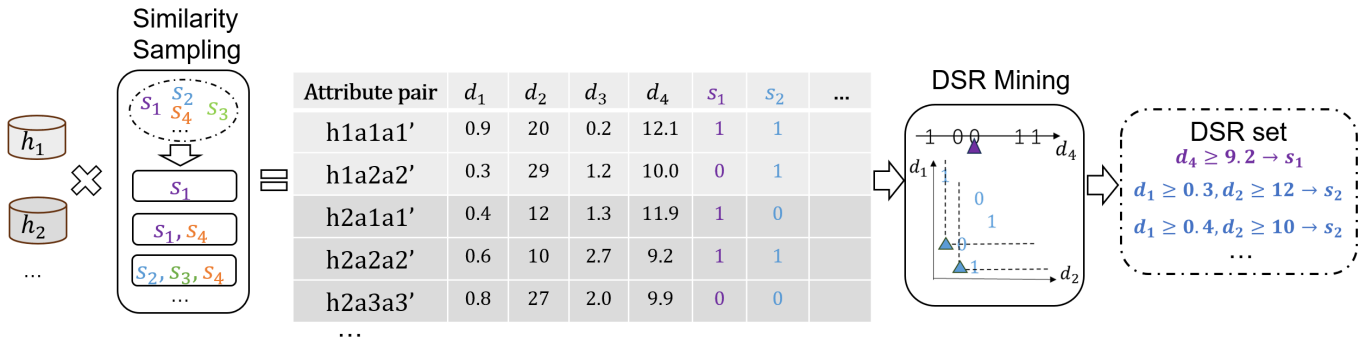
$$r: \bigwedge_{d_i \in D' \subseteq D} d_i \geq \delta_i \rightarrow S_j \tag{2}$$

For example, a valid DSR can be written as:

$$(length \geq 10) \wedge (entropy \geq 2.92) \rightarrow Levenshtein$$

Given an entity pair (t_1, t_2), for each attribute A_i, DSR first checks whether its `length` ≥ 10, and then checks whether its `entropy` ≥ 2.92. If all conditions are met, DSR computes the Levenshtein Similarity between $t_1[A_i]$ and $t_2[A_i]$ as the feature. Rules in the DSR set will be traversed, and all similarity functions that meet the *description* constraints will be added to the feature set for the current attribute.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.



Attribute pair	d_1	d_2	d_3	d_4	s_1	s_2	...
h1a1a1'	0.9	20	0.2	12.1	1	1	
h1a2a2'	0.3	29	1.2	10.0	0	1	
h2a1a1'	0.4	12	1.3	11.9	1	0	
h2a2a2'	0.6	10	2.7	9.2	1	1	
h2a3a3'	0.8	27	2.0	9.9	0	0	

Fig. 3: The DSR Mining workflow.

The rules in Magellan and AutoML-EM are special cases of DSR. Magellan uses the `length` as the *description* and sets the threshold δ with heuristics. Similarly, AutoML-EM provides rules with simpler DSR antecedents to suggest similarity functions for attributes. It categorizes attributes (including numerical, string, and boolean) and recommends extensive sets of similarity functions for each attribute category.

C. Problem Definition

Due to the vast search space formed by high-level statistics and parameters, manually crafting DSR is intractable. Therefore, we focus on *DSR mining* as follows:

Problem 1. ***Description-Similarity Rule Recommendation:*** *Given an FE task, a performance threshold θ, the attribute description set D and the similarity function set S, find the most* ***representative*** *rule set R where:*

$$minimize \quad |R| \tag{3}$$
$$R = \{r | r : \bigwedge_{d_i \in D' \subseteq D} (d_i > \delta) \rightarrow S_j\}, p(M(S)) > \theta$$

where S is the union of similarity function S_j recommended by rules R, and $p(M(S))$ is the performance of model M with S on the FE task. The representative rule set R will be discussed in Section IV-A.

A brute-force traversal in the search space of DSR would require $2^{|d|} \times dom(d_i) \times |S|$ model retraining, where $dom(d_i)$ is the domain of valid value of description d_i and is mostly exhaustive to explore.

D. Solution Overview

In this section, we introduce the architecture of the DSR Mining Framework. Firstly, the Similarity Sampling Module retrains some EM models offline on historical EM datasets, where each trial uses a specific feature set. After that, the DSR Mining Module collects the performance of these trials and derives a DSR set from them. The details are as follows.

Similarity Sampling Module. To guarantee good performance for features given by DSR, a thorny task is to estimate the expected contribution of each similarity. A rough approximation for the expected performance using a similarity can result in a low-quality DSR set. Therefore, to ensure a controllable error of approximation with a simple strategy, the Similarity Sampling module develops a multi-armed bandit-based sampling algorithm to evaluate the similarity function regarding threshold θ.

DSR Mining Module. The first module generates a collection of Attribute-Description-Similarity (gray table in Figure 3, referred to as **ADS** below, where X_i is d_i, y_j is s_j). As the core step of our framework, the DSR mining module finds representative rules in meta-learned experience. Both single and multiple DSR cases follow a pruned sorting-enumeration procedure, reducing the time cost of online computation (model retraining on the target dataset). Description-Similarity Rules can be used for online similarity recommendation.

III. Similarity Sampling

In Problem 1, the EM model using similarities recommended by DSR is *expected* to perform as well as θ, so a primary question is: **what is the impact of each similarity on various EM tasks?** In other words, can we expect EM model performance higher than θ by using a similarity on a specific EM task? Direct computation of *expectation* requires $O(2^{|S|})$ EM model retraining. To reduce the computation overhead, we define the problem formally in Section III-A, introduce sampling-based model retraining strategy in Section III-B, and elaborate the budget allocation and combination method in Section III-C.

A. Thresholding Similarity Expectation

In this section, we formally define the problem of comparing similarity performance expectation with threshold θ. The Thresholding Similarity Expectation (TSE) problem decides whether the use of each similarity can give a performance expectation higher than θ on a target dataset. On a given EM task and model, TSE computes y_j for each S_j as follows.

$$y_j = \begin{cases} 0, & E(p(S_j)) = \frac{1}{|2^{|S|-1}|} * \sum_{t \subset S, S_j \in t} p(t) < \theta \\ 1, & E(p(S_j)) = \frac{1}{|2^{|S|-1}|} * \sum_{t \subset S, S_j \in t} p(t) \geq \theta \end{cases}$$

where variable $p(S_j)$ is the performance of the EM model on the given task using any subset containing similarity S_j.

The core of TSE is computing $E(p(S_j))$, which requires exponential retraining of EM models online. $E(p(S_j))$ is actually

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

an aggregation function avg. Inspired by recent advances in approximation aggregation [46], we model similarities in each EM data set as a multi-armed bandit (MAB).

Note that we do not intend to contribute to the design of novel sampling strategies. Instead, we contribute to proposing simple yet effective solutions never explored in previous studies, adapting Multiple Arm Identification (MAI) strategies for TSE and combining MAI solver with meta-learning.

B. Single MAB Sampling

We discuss how to use MAB to reduce the retraining of the EM model.

Multi-armed Bandit (MAB). MAB is a slot machine with K arms$\{1, .., K\}$. An agent decides which arm to pull in each of the n total rounds. When pulled, arm i returns a random reward from an unknown distribution D_i specific to arm i.

Multiple Arm Identification (MAI). Unlike the traditional goal where the agent maximizes the total reward or minimizes the regret, in Multiple Arm Identification (MAI), the agent identifies a subset of the arms $\{J_1, ...\} \subset \{1, .., K\}$ corresponding to a specified criterion [47].

Model as MAI. Choosing similarity for each attribute pair can be viewed as Multiple Arm Identification, where each arm is a similarity, and the reward distribution is made up from EM performance using this similarity. According to the setting of the TSE problem, we set the criterion as a constraint $\mu_{J_i} \geq \theta$.

The CSAR solution. Deriving the best algorithm for a specific case of MAI is an open problem [48]. We adopt the SAR algorithm for our problem as shown in Algorithm 1. Each arm represents a similarity measure. Selecting each of the remaining arms $i \in \mathcal{A}_K$(line 4) means sampling $n_k - n_{k-1}$ different subsets of similarities that includes the i-th similarity. The performance of the EM model is then evaluated using these feature sets, and the results are recorded to update μ_{i_k} using current expectation $E(p(S_i))$. At the end of each iteration, the algorithm decides whether to accept the k-th arm (line 6 to line 9) with largest distance to θ (line 5), deciding its expectation higher than θ.

Algorithm 1 CSAR

Input: MAB $\{1, ..., K\}$, threshold θ, budget n
Output: The accepted arms $\mathcal{A}_K$

1: $n_0 = 0; \mathcal{A}_K = \{\}; \mathcal{A}_1 = \{1, ..., K\}$
2: **for** $k = 1, 2, ..., K-1$ **do**
3: $\quad n_k = \left\lceil \frac{1}{\overline{log}K} \frac{n-K}{K+1-k} \right\rceil$
4: $\quad$ select each arm $i \in \mathcal{A}_K$ for $n_k - n_{k-1}$ rounds
5: $\quad i_k \in \mathsf{argmax}_{i \in \mathcal{A}_k} |\mu_{i,k} - \theta|$
6: $\quad$ deactivate arm i_k by setting $\mathcal{A}_{k+1} = \mathcal{A}_k \setminus \{i_k\}$
7: $\quad$ **if** $\mu_{i_k} \geq \theta$ **then**
8: $\quad\quad$ accept arm i_k
9: $\quad$ **end if**
10: **end for**
11: **return** $\mathcal{A}_K$

Error and Time Cost. The probability of error satisfies:

$$e_n \leq 2K^2 exp(-\frac{n-K}{2\overline{log}K * H})$$

where $H = \max_{i \in \{1,..,K\}} i * (|\mu_i - \theta|)^{-2}$, $\overline{log}K = \frac{1}{2} + \sum_{i=2}^{K} \frac{1}{i}$. (See [45] for detailed proof)

If we want $e_n \leq \delta$, CSAR Algorithm needs to train $2\overline{log}K * H * ln(\frac{2K^2}{\delta}) + K = O(ln(\frac{1}{\delta}))$ models online (line 4), which is too high a cost as one always wants to minimize error δ.

C. Meta-learned Multi-arm Identification

To improve the efficiency of CSAR, we introduce the idea of meta-learning to form a pre-sampling solution.

Meta-Learning. Meta-learning, or learning to learn, can capture the task structure to enable few-shot learning [49]. The TSE problem prepares trials on historical tasks (ADS collection) for mining DSR set adaptable to a new EM task, which well fits the meta-learning idea [50]. Therefore, we propose to combine meta-learning with CSAR. We run multiple trials on historical EM datasets with different combinations of similarities offline using CSAR and combine the performance of related trials online with the meta-feature below.

Meta Feature-based similarity expectation. Using Meta Feature in [51], the expectation f_j of similarity S_j on attribute pair A on dataset D is as follows.

$$f_j = \frac{\sum_{t \subset T, S_j \in t, A' \in D} cos < \vec{D}, \vec{D'} > * p(t, A')}{\sum_{t \subset T, S_j \in t, A' \in D} cos < \vec{D}, \vec{D'} >} \quad (4)$$

Here $\vec{D}$ is meta-feature of dataset D, and $p(t)$ is the performance p on trial t. When computing f_j for similarity S_j on attribute A, only trial $t \in T$ containing S_j are used.

Model as Meta-learned Multi-Arm Identification. Equation 4 is a linear projection, so we model the process as a Meta-learned Multi-Arm Identification problem. That is, given a set of historical k-armed bandit $\{\mathsf{MAB}_1, ..., \mathsf{MAB}_a\}$, an invisible bandit MAB, a weight vector $w_i, 1 \leq i \leq a$, and a constraint $f \geq \theta$, the aim is to find an optimal sampling strategy on $\{\mathsf{MAB}_1, ..., \mathsf{MAB}_a\}$ within a fixed budget N, where probability of error e_N is minimized.

The Pre-Sampling Solution. We already give CSAR algorithm for each single $\mathsf{MAB}_i \in \{\mathsf{MAB}_1, ..., \mathsf{MAB}_a\}$, so the remaining effort is sample combination. We propose an algorithm called P-CSAR (Algorithm 2), combining the offline bandit samples with the online bandit.

In Algorithm 2, lines 1 to 4 describe the offline stage of P-CSAR, where line 3 runs the CSAR algorithm for $\mathsf{MAB}_i, i \leq a$ with a uniformed budget. In the online stage (lines 5-10), the algorithm uses meta-learning to estimate each similarity expectation of MAB (line 6), and select those larger than θ as the result (line 8).

We analyze the error probability of P-CSAR to meta-learning as follows, where "error" means xxx. The error probability of our meta-learning strategy can be derived from the generalization error of linear regression [52], [53].

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 P-CSAR

Input: Multi-armed Bandits set $\mathsf{MAB} = \{\mathsf{MAB}_1, ..., \mathsf{MAB}_a\}$, target Multi-armed Bandit MAB_0, threshold θ, budget n
Output: The accepted arm set $\mathcal{J} \subset \mathsf{MAB}_0$

```
1: F ← ∅, J ← ∅;
2: for MAB_i ∈ MAB do
3:    F ← F ∪ CSAR(MAB_i, θ, n/a)
4: end for
5: for A_j ∈ MAB_0 do
6:    compute f_j in Equation 4 with F
7:    if f_j > θ then
8:       J ← J ∪ A_j
9:    end if
10: end for
11: return J
```

Proposition 1. *The error probability of P-CSAR satisfies:*

$$e_N \leq 2aK^2 exp(-\frac{n-aK}{2a\overline{log}K * H(a)}) \tag{5}$$

where $H = \max_{i\in\{1,..,K\}} i * (|\mu_i - \theta|)^{-2}$, $H(a) = \max_{a\in\mathsf{MAB}} H(a)$.

The proof of Propositions are omitted for simplicity and can be found in [45].

The uniform budget is a heuristic setting, as the global optimal setting is impractical. In [45], we give the analytic expression of the budget allocation for the minimized error probability, but it requires prior knowledge such as the maximum difference of MAB expectation from the decision boundary $H(a)$ in Equation 5, while it can only be known after sufficient sampling rounds.

Time Cost Analysis. From Proposition 1 we have:

$$2aK^2 exp(-\frac{N-aK}{2a\overline{log}K * H(a)}) = \delta$$
$$N = 2a\overline{log}K * H(a) * ln(\frac{2aK^2}{\delta}) + aK$$

This implies that P-CSAR requires $N = an + 2a\overline{log}K * H * lna = O(n)$ offline cost to obtain an error bound comparable with the online scenario.

IV. Description-Similarity Rules

In this section, we propose our DSR set discovery algorithm using *descriptions* and the approximated similarity expectation above. As an online process, the computational cost is highly restricted, but the DSR search space is overwhelming. We briefly introduce how to use the Description-Similarity Rule set for feature engineering (Section IV-A), discuss our pruning heuristic and traversal algorithm designs of single (Section IV-B) and multiple (Section IV-C) DSR mining algorithm to accelerate DSR mining.

A. Using DSR for Feature Engineering

As shown in the example of Figure 3, given a DSR set and an EM task, it is straightforward to generate EM features. That is, simply compute four descriptions for each attribute pair and select similarities according to the antecedent of DSR. The technical challenge is to discover a "small and sweet" DSR set given historical data, where "small" guarantees explainability, and "sweet" ensures good EM performance using them.

A "sweet" DSR set is already guaranteed by sup and θ. A "small" DSR set is a minimal cover of DSR to ensure explainability, as an overwhelming and redundant DSR set will add to the fuzziness of feature engineering. Actually, the DSR set compression relies on the partial dominance of DSR candidates, which is shown in the following proposition.

Proposition 2. *If DSR* $d_1 < \delta_1, d_2 < \delta_2, ..., d_k < \delta_k \rightarrow S_i$ *holds, for* $\delta_1 < \delta_1', \delta_2 < \delta_2', ..., \delta_k < \delta_k'$ *, DSR* $d_1 < \delta_1', d_2 < \delta_2', ..., d_k < \delta_k' \rightarrow S_i$ *must hold.*

Proposition 2 is designed for DSR set compression, so the optimization goal of DSR compression is minimal solution in single DSR (Problem 2 in Section IV-B) and Pareto Optimal in Multiple DSR (Problem 3 in Section IV-C).

B. Single DSR Mining

We start with a discussion of a single DSR, a special set of DSR where only one description is used as the antecedent.

$$d(A_i, A_j) > \delta \rightarrow S_i(A_i, A_j),$$

Inspired by rule mining in data cleaning literature [5], [54]–[57], we define our goal as the following problem.

Problem 2. ***Single Description-Similarity Rule Mining:*** *Given an ADS collection* $\{c|c =< X_i, y_j >, 1 \leq i \leq N\}$, *supporting threshold* sup*, the Single Description-Similarity Rule Mining problem aims to find:*

$$\delta^* = min_{\delta\in D(X_i)}\delta \tag{6}$$
$$s.t. S_i = \frac{|X_i > \delta, y_j > \theta|}{|X_i > \delta|} \geq \text{sup}$$

Here y is a binary value generated from the similarity expectation thresholding process in Section III. For each possible value $c \in c.X_i$, the Brute-Force method needs to split $N-1$ data points in two groups, getting δ^* in $O(N*(N-1)) = O(N^2)$ time. As shown in Example 3, a Sorting-Enumeration procedure can reduce redundant data splitting of the Brute-Force method.

Example 3. *Take Figure 4 as an example, supposing* sup $= 0.6$*, to find the minimal solution, we first sort the data by* X_i *and enumerate the sorted list in ascending order. The first satisfied solution* $\delta = c.X_i$ *must be* δ^* *(the green triangle in Figure 4). We call this procedure Naive SE (NSE), that is, a full sorting (any existing sorting algorithm) followed by an enumeration (lines 14-19 of Algorithm 3).*

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 PRUNED SE

Input: An ADS collection $\mathcal{B} = \{c|c =< X_i, y_j >, 1 \leq i \leq N\}$, supportive threshold sup
Output: The minimal threshold δ^* for X_i

1: $\mathcal{B} \leftarrow \{\mathcal{B}\}$
2: **while** $\max_{b \in \mathcal{B}} |b| > \frac{N}{logN}$ **do**
3: $\quad \mathcal{B} \leftarrow Partition(\mathcal{B})$
4: $\quad$ **for** new buckets $[b_i, b_{i+1}] \in \mathcal{B}$ **do**
5: $\quad\quad$ **if** $\frac{evi_i}{obs_i} \geq$ sup **then**
6: $\quad\quad\quad \mathcal{B} \leftarrow MergeAllRight(b_i)$
7: $\quad\quad$ **end if**
8: $\quad\quad$ **if** $l_-(b_i, \text{sup}) + r_+(b_{i+1}, \text{sup}) \geq |b_i|$ **then**
9: $\quad\quad\quad \mathcal{B} \leftarrow Merge(\mathcal{B}, b_i)$
10: $\quad\quad$ **end if**
11: $\quad$ **end for**
12: **end while**
13: $SortBuckets(\mathcal{B})$
14: $\mathcal{B} \leftarrow points(\mathcal{B}), obs \leftarrow |\mathcal{B}|, i \leftarrow 0, evi \leftarrow Ones(\mathcal{B})$
15: **for** $0 \leq i \leq |\mathcal{B}|$ **do**
16: $\quad$ **if** $\frac{evi_i}{obs_i} \geq$ sup **then**
17: $\quad\quad$ **return** $\mathcal{B}(i)$
18: $\quad$ **end if**
19: **end for**
20: **return** $null$

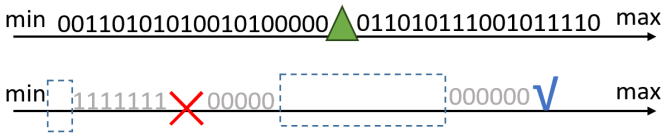


Fig. 4: Single DSR Mining

NSE can reduce evi and obs update cost during enumeration from $O(N)$ to $O(1)$ for each candidate. Therefore, the time complexity bottleneck of NSE is its $O(NlogN)$ fully sorting before enumeration. As data augmentation methods like [14] can enrich the size of ADS collection N to a large number, we propose a pruning heuristic to accelerate NSE.

The conclusion of Proposition 3 is demonstrated in the example from Figure 4. Given an unsatisfied candidate (red cross), δ^* won't appear in its neighborhood. Symmetrically, given a satisfied candidate (blue check), δ^* won't appear in its left neighborhood or any candidate on its right.

Proposition 3. *If the support of the r-th largest number p in C is $S(p) = \frac{evi(p)}{obs(p)}$, the following two propositions hold:*

(1) If $S(p) <$ sup, $rank(\delta^*) \notin R(p) = [r - l_-(p, \text{sup}), r + r_-(p, \text{sup})]$

(2) If $S(p) \geq$ sup, $rank(\delta^*) \notin R(p) = [r - l_+(p, \text{sup}), \infty)$

where we denote

$$l_-(p, \text{sup}) = \lfloor \frac{\text{sup} * obs(p) - evi(p)}{1 - \text{sup}} \rfloor$$

$$r_-(p, \text{sup}) = \lfloor obs(p) - \frac{evi(p)}{\text{sup}} \rfloor$$

$$l_+(p, \text{sup}) = \lfloor \frac{evi(p)}{\text{sup}} - obs(p) \rfloor$$

to slightly shorten the notation.

We give an algorithm called Pruned SE (PSE) to accelerate the single DSR mining with pruning. PSE follows the divide-and-conquer idea to randomly divide data into groups, i.e., buckets. Pruning with Proposition 3 can be easily performed by merging buckets into "heavy points" (line 6 and line 9) so that the pruned data points in such buckets no longer need to be sorted (line 13). Also, constraining the maximal size of bucket(line 2) to $\frac{N}{logN}$ reduces the computation cost bottleneck of sorting in line 22 from $O(NlogN)$ to $O(N)$.

C. Multiple DSR Mining

The expressiveness of a single DSR is insufficient. The real challenge is to mining DSR with multiple descriptions used as an antecedent:

$$d_1(A_i, A_j) > \delta_1, .., d_k(A_i, A_j) > \delta_k \rightarrow S_i(A_i, A_j)$$

For multiple DSRs, the target subset according to Proposition 2 is based on the definition of Pareto Dominance [58].

Definition 4. *Pareto Dominance: Consider a dataset D with d tuples that have the preference function in each attribute less than(<). A point t Pareto dominates another point u, denoted by $t \succ u$, if and only if $\forall i, t_i \leq u_i$ and $\exists j, t_j \leq u_j$. A point t is in the Pareto Optimal set $P(D) \subset D$ if and only if there does not exist any other point u in the dataset that Pareto dominates it, that is, $t \in P(D) \iff \exists u \in D, u \succ t$.*

Specifically, our goal is defined as the following problem.

Problem 3. *(Multiple Description-Similarity Rule Mining): Given an ADS collection $\{c|c =< X_1, X_2, ..., X_k, y_j >, 1 \leq i \leq N\}$ and supportive threshold* sup*, the Multiple Description-Similarity Rule Mining problem aims at finding the maximal set Δ' satisfying:*

$$\forall d \in \Delta, S_i(d) = \frac{|X_1 \geq \delta_1, ..., X_k \geq \delta_k, y_j \geq \theta|}{|X_1 \geq \delta_1, ..., X_k \geq \delta_k|} \geq \text{sup} \quad (7)$$
$$\Delta' = P(\Delta)$$

Problem 3 first finds all the satisfied solutions and then uses the Pareto Optimal Set of the solutions as the DSR set.

Again, the Brute-Force algorithm traverses each possible solution and verifies them with partition. As shown in Figure 5(a), the Pareto Optimal solution can appear outside the ADS collection. Although the partition for each candidate

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

still costs $O(N)$ time, there are $O(N^k)$ possible solutions according to Proposition 4.

Proposition 4. *If the input size of Problem 3(size of ADS collection) is N, the size of solution space is $O(N^k)$.*

The time complexity of a Brute-Force traversal is $O(N^{k+1})$ as $O(N^k)$ data partition is needed. We design Algorithm 4 accelerating Brute-Force traversal by one order of magnitude.

Algorithm 4 PRUNED-IES

Input: An ADS collection $\mathcal{C} = \{c | c =< X_1, X_2, ...X_k, y_j >, 1 \leq i \leq N\}$, supportive threshold sup
Output: Description-Similarity Rules $Rules$

1: $Candidates \leftarrow \{X_1\} \times \{X_2\} \times ... \times \{X_k\}$
2: **if** k=2 **then**
3: $Rules \leftarrow$ PRUNEDSE$(\mathcal{C}, \text{sup})$
4: **else**
5: $Rules \leftarrow$ PRUNED-IES$(\mathcal{C}\{k-1\}, \text{sup})$
6: **end if**
7: $Candidates \leftarrow$ *DimensionalPruning*$(Rules)$
8: **for** $j \in Candidates$ **do**
9: compute evi_j, obs_j with 8
10: **if** $\frac{evi_j}{obs_j} \geq$ sup **then**
11: $Rules \leftarrow Rules \cup j$
12: **end if**
13: **end for**
14: **return** $Rules$

This is a recursive algorithm (lines 2-6). Dimensional-Pruning can delete redundant points [57] according to Proposition 5 (line 7). Stratified-Sort (line 8) prepares data for the enumeration (lines 9-15).

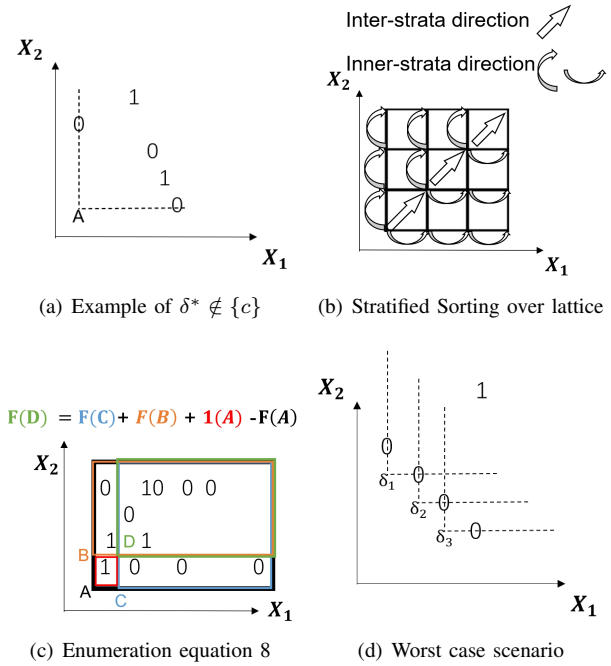
Proposition 5. *If DSR $d_1 > \delta_1, d_2 > \delta_2, ..., d_{k_1} > \delta_{k_1} \rightarrow S_i$ holds, the DSR with $d_{k_2} - dk_1$ extra antecedent $d_1 > \delta_1, d_2 > \delta_2, ..., d_{k_1} > \delta_{k_1}, ..., d_{k_2} > \delta_{k_2} \rightarrow S_i$ must hold.*

Dimensional Pruning. The Dimensional Pruning in line 7 takes advantage of the inclusion-exclusion principle:

$$|\cup_{i=1}^{n} A_i| = \sum_{k=1}^{n} (-1)^{j-1} \sum_{1<i_1<i_2<...i_k \leq n} |A_{i_1} \cap A_{i_2} \cap ... \cap A_{i_k}|$$

Suppose a hyper-cube $Cube$ with a vertex on the grid generated by n samples in a k-dimensional space, and the vertex closest to the coordinate origin is O'. By setting $n = k + 1$, $\vec{a_i}$ is the direction vector of the hyper-cube on the i-th dimension, $A_i = \{c | O' + \vec{a_i} \succ c\}$, and $A_0 = \{Cube\}$, we have the following equation. The example in Figure 5(c) demonstrates this using $ABCD$ as a hypercube.

$$F(\vec{0}) = F(Cube) + \sum_{i_1=1}^{k} F(\vec{a_i}) - \sum_{i_1=1}^{k} \sum_{i_2=i_1+1}^{k} F(\vec{a_i} + \vec{a_j}) + ... + (-1)^{k-1} \sum_{i_1=1}^{k} ... \sum_{i_k=k}^{k} F(\vec{a_{i_1}} + \vec{a_{i_2}} + ... + \vec{a_{i_k}}) \quad (8)$$



(a) Example of $\delta^* \notin \{c\}$ (b) Stratified Sorting over lattice

(c) Enumeration equation 8 (d) Worst case scenario

Fig. 5: Two-dimensional case of Multiple DSR Mining

where $F(.)$ can be either $obs(O' + .)$ or $evi(O' + .)$. Specifically, $F(Cube)$ is $obs(Cube)$ or $evi(Cube)$.

To demonstrate Equation 8, we give an easy-to-understand two-dimensional example.

Example 4. *Suppose a two-dimensional case in Figure 5(c). The zeros and ones refer to trials with performance higher and lower than θ, respectively. We elaborate Equation 8 with square $ABCD$. Here, $F(A), F(B), F(C)$, and $F(D)$ are the cardinality of the ones and zeros in the black, orange, blue, and green rectangle, respectively. Supposing $F(A), F(B), F(C)$ have already been computed, we can trade the $O(1)$ computation of $\mathbf{1}(A)$(point A scan) for $O(N)$ computation of $F(D)$. A traversal following the arrow in Figure 5(b) can guarantee that the above trade-off always holds, which can be applied for all candidate points in the green frame, reducing time complexity from $O(N^3)$ to $O(N^2)$.*

The example shows that computing $F(.)$ for "outer points" on all $(k-1)$-dimensional hyperplane containing O' is enough for this problem, and the remaining "inner points" are computed with Equation 8. The key computational reduction is trading an $O(N^{k-1})$ term $F(\vec{a_{i_1}} + \vec{a_{i_2}} + ... + \vec{a_{i_k}})$ for an $O(1)$ term $F(Cube)$.

Proposition 6. *The time complexity of Algorithm 4 is $O(N^k)$.*

Finally, we discuss the inherent complexity of Problem 3 in the worst case. Note that this complexity is a loose bound since it is only the complexity of printing the solution.

Proposition 7. *If the input size of Problem 3 (size of ADS collection) is N, the inherent complexity of Problem 3 is*

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

$O(N^{k-1})$ *in the worst case.*

Proposition 6 and 7 show that **Pruned-IES is at most one order of magnitude slower than the fastest algorithm**.

V. EXPERIMENTS

We conduct extensive experiments on real-world EM benchmarks to evaluate the performance of DSR. In this section, we first describe experimental settings in Section V-A, followed by an end-to-end performance evaluation in Section V-B, a detailed ablation study in Section V-C, and a sensitivity analysis in Section V-D. Empirically, DSR outperforms the rule-based feature engineering methods and approaches the performance of deep-learning methods with much lower computation overhead.

A. Experimental Settings

Datasets. We use two popular [1], [6] benchmark datasets: Magellan [23] and WDC [59]. For datasets in the Magellan Repository, we split each dataset and use 80% for training and 20% for testing. To examine the scalability of DSR, we bring in WDC following recent work [1], [60]. WDC records entities from 4 categories of products: computers, cameras, watches, and shoes. Each WDC dataset has four different sizes including small, medium, large, and xlarge.

We set the default parameters as follows: support threshold $\mathrm{sup} = 0.33$, and similarity expectation threshold $\theta = 0.83$. The value of the budget for historical sampling is initialized as 200, according to available computing resources. The sensitivity analysis can be found in Section V-D.

Baseline. To validate the effectiveness of DSR, we select six EM solutions for comparison. Furthermore, we choose two feature importance evaluation methods to compare the performance of P-CSAR.

• AutoML-EM [6] is a machine learning-based approach that generate data labels with active learning and selects features based on AutoML.

• Magellan [23] is a learning-based and end-to-end EM system on structured data. It keeps the developer in the loop in every step and provides guidance for developing an end-to-end EM solution.

• Ditto [1] is a deep learning-based subsequent state-of-the-art EM solution which injects Domain-Knowledge to language models.

• Rotom [14] is a data augmentation framework for EM, which outperforms DeepMatcher [61], RoBERTa-based models [62] with limited training samples.

• Sudowoodo [13] is the state-of-the-art data preparation framework for EM tasks. Based on self-supervised learning, Sudowoodo outperforms DeepMatcher [61], Ditto [1], and Rotom [14] on certain datasets.

• DSR-KNN [26] represents entity pairs with DSR-recommended features, assigns the majority class among k closest neighbors as the matched / unmatched decision, following the usage of a previous work on data quality [63].

• L2 norms [27] is a popular feature selection method. Based on Lasso regression [64], it adds penalty terms to the loss function and removes irrelevant similarities for EM.

• SHAP [28] and LIME [29] are two tools for interpreting predictions of machine learning models. SHAP assesses the contribution of features across the entire dataset, while LIME investigates the contribution of features on each entity pair.

Evaluation Metric and Environment. We use F1 score as the metric. The default time unit is second. All experiments were run on an Ubuntu 20.04 server with an Intel Xeon E5-2678 v3 (2.50GHz) CPU and 32GB memory, Tesla M40 GPU.

B. End-to-End Performance

We conducted end-to-end comparisons on Magellan Repository and WDC datasets. Our results are shown in Table II.

Comparison with Machine-Learning based Solutions. DSR significantly outperforms the reported performance [6] of state-of-the-art machine learning solutions.

As shown in Table II, DSR demonstrates superior performance compared to Magellan and AutoML-EM, with 3270 seconds saved on average compared to AutoML-EM.

Comparison with Deep-Learning-based Solutions. We compare DSR to the mainstream and SOTA deep learning solutions. We apply the same experiment configuration as the SOTA solution Sudowoodo. Our results of deep learning-based baselines are consistent with [13].

As shown in Table II, DSR has outperformed existing deep-learning solutions on most datasets. These solutions often struggle when training data is insufficient. However, on large-scale datasets like WDC-xlarge, which contains up to 21 million entity pairs, Ditto achieves superior results. This can be attributed to its ability to model complex features effectively, which becomes increasingly advantageous in data-rich scenarios.

DSR or Deep Learning-based EM? Deep Learning-based EM models learn complex features by implicit non-linear transformations, but are sensitive to noise [65]–[67] and thus require numerous training data like WDC-xlarge to filter out such noise [68]. Insufficient datasets often fail to support these data-hungry models to learn high-level and high-quality features, resulting in suboptimal performance [69]. EM task is built on tabular data, which has a well-defined schema, and structure for feature extraction and is friendly for machine learning. Combining DSR, the Random Forest model can effectively learn decision rules from smaller datasets. DSR outperforms mainstream feature selection methods (Section V-C), thus achieving better performance compared to deep learning-based EM models on smaller and medium datasets. As for large datasets where deep learning methods beat DSR, a possible improvement is introducing Pretrained Language Model-generated embeddings as a similarity candidate [70]. Using such embeddings as predicates, we can potentially extend the flexibility of DSR. We leave this as future work.

Comparison with KNN. We attempt to prove the design choice of models in the DSR mining module, answering the question: can simple Machine Learning models solve the

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Effectiveness and efficiency evaluation of end-to-end solutions on EM datasets, including Magellan Repository and WDC dataset (5 versions). We reproduce all baselines using the available source code. We use × to represent cases where the source code does not support testing for this dataset.

Data Source	Dataset	AutoML-EM		Magellan		Ditto		Sudowoodo		Rotom		DSR-KNN		DSR-L2 norms		DSR		Δ(vs. AutoML-EM)
		F1	time	F1	time	F1	time	F1	time	F1	time	F1	time	F1	time	F1	time	F1
Magellan Repository	Fodors-Zagats(FZ)	100	3683	100	32.2	95.2	53	100	538	97.7	57.18	100	27.01	93.02	27.08	**100**	33.9	(+0)
	iTunes-Amazon(IA)	71.7	3686	52.1	33.3	62.3	135	87.27	659	92.9	95.33	58.18	29.01	66.67	29.15	**100**	28.76	(+28.3)
	s-iTunes-Amazon	96.4	3681	91.2	38.7	87.2	26	94.34	802	94.5	98.06	94.59	32.01	94.73	32.12	**100**	28.58	(+3.6)
	Beer(BR)	93.6	3686	78.8	34.2	45.2	53	63.63	373	52.2	44.43	80	29.01	88.89	29.08	**94.4**	25.44	(+0.8)
	DBLP-GoogleScholar(DG)	88.3	3708	81.2	98.6	89.9	2451	×	×	88.5	340.85	76.93	100.13	88.63	100.24	**90.57**	116.4	(+2.27)
	DBLP-ACM(DA)	94.7	3698	88.3	70	**96.2**	2704	×	×	90.5	177.15	87.66	63.06	93.22	63.96	94.96	58	(+0.26)
	s-DBLP-GoogleScholar	94.5	3705	92.3	111.4	90.16	3052	89.51	2351	84.8	298.56	63.51	115.19	94.03	115.98	**95.22**	104	(+0.72)
	s-DBLP-ACM	98.5	3691	98.4	67.7	96.9	1187	94.13	2403	90.4	165.27	90.14	70.1	97.67	70.98	**98.98**	66.1	(+0.48)
	Amazon-Google(AG)	56	3695	49.6	45.7	58.67	251	×	×	57.82	103	46.33	48	51.04	48.97	**62.31**	43.3	(+6.31)
	Walmart-Amazon(WA)	45.16	3668	36.7	52.5	49.7	311	32.99	1440	**71.43**	154.18	27.56	57.05	38.76	57.28	51.16	55	(+6)
	s-Walmart-Amazon	75.74	3667	71.9	54.2	65	577	30	1510	68.51	124.83	57.03	63.1	75.06	63.65	**76.4**	55.3	(+0.66)
WDC-xlarge	Computers	71.54	3763	65.97	26	**87.6**	17340	51.65	4560	24.2	1889.57	55.54	815	67.02	834.78	74.62	1002	(+3.08)
	Cameras	69	3948	64.91	26.1	**93.09**	3954	43.26	3180	17.4	1247.14	55.93	1336	66.89	1354.17	72.94	1273	(+3.94)
	Shoes	57.35	3765	45.65	18.8	**84.72**	4230	30.06	6760	50.2	1867.3	35.4	680	54.93	695.13	61.72	667	(+4.37)
	Watches	83.25	3949	78.53	44.37	**91.96**	4114	53	12120	35.2	1152.78	70.6	1374	79.77	1398.6	84.82	1406	(+1.57)
	All-xlarge	71.91	4592	65.43	144.7	**87.48**	26051	62.67	22140	20.3	5820.23	52.74	4235	66.53	4335	74.79	4216	(+2.88)
WDC-large	Computers	73.55	3693	67.3	19	**89.91**	2487	49.02	2436	25.3	948.51	56.96	426	70.97	442.62	76.59	387	(+3.04)
	Cameras	67.34	3760	63.04	12.74	49.36	1093	32.72	1857	21	659.46	55.78	617	63.88	625.6	**71.52**	552	(+4.18
	Shoes	63.68	3691	52.96	9.3	47.71	1254	58.9	2955	43.9	857.54	43.4	365	62.7	372.29	**70.4**	378	(+6.72)
	Watches	78.26	3744	73.72	20.01	49.15	1475	69.58	2179	27.2	649.46	62.38	578	74.75	585.5	**80.52**	599	(+2.26)
	All-large	70.33	4082	62.41	67	**84.33**	5345	54.42	5916	19.8	2796.59	52.57	1847	67.76	1886.78	73.18	1688	(+2.85)
WDC-medium	Computers	72.19	3638	68.36	9.75	46.24	494	75.11	1273	54.1	332.6	57.39	138	69.97	140.6	**73.18**	165	(+0.99)
	Cameras	60.77	3654	55.86	4.17	50.62	330	60.18	1269	23.7	246.85	54	162	62.44	164.1	**64.61**	201	(+3.84)
	Shoes	62.24	3636	54.22	3.62	48.04	180	61.82	1111	45.5	297.72	53.4	112	63.58	113.77	**74.05**	168	(+11.81)
	Watches	69.79	3646	66.7	5.32	51.1	436	59.96	1057	33.4	265.3	52.79	148	65.18	151.13	**67.55**	211	(-2.24)
	All-medium	67.15	3724	59.41	20.44	49.11	1366	58.49	2157	22.9	757.07	50.2	454	65.5	461.87	**69.19**	455	(+2.04)
WDC-small	Computers	78.49	3631	68.36	2.63	53.92	218	81.3	1341	35.8	202.29	66.42	64	72.32	67	**78.96**	112	(+0.47)
	Cameras	72.13	3631	55.86	2.07	47.57	164	67.84	1295	23.5	177.48	55.62	73	61.96	73.59	**74.03**	123	(+1.9)
	Shoes	69	3626	54.23	1.88	53.27	163	69.23	480	53.6	195.55	53.84	61	66.32	61.43	**72.27**	117	(+3.27)
	Watches	69.02	3629	65.8	3.75	50.14	218	73.48	1734	42.5	171.42	57.01	65	66.06	65.83	**75.37**	116	(+6.35)
	All-small	70.47	3661	63.13	9.2	50.26	545	58.49	1877	31.9	345.84	57.17	172	70.43	173	**74.49**	219	(+4.02)
All Above	average	74.58	3743	67.49	35.14	67.8	2654	62.97	3135	49.7	726	60.68	463	71.63	472	**78.35**	473	(+3.77)

problem? We represent entity pairs with similarities generated by DSR and choose KNN [26] as an EM model. As shown in Table II, without DSR mining, KNN on the description space became excessively informative, leading to poor performance compared with Random Forest in both F1 score and time.

Comparison with Feature Selection Methods. We aim to demonstrate the superiority of the DSR-based feature selection method compared to another feature selection method, L2 norms. Our results are shown in Table II. DSR obtains better performance compared to L2 norms within less time.

Takeaway: On most benchmark datasets, DSR achieves the best average performance in all EM baselines in Section V-A, within minimal runtime. The similarities generated by DSR beat general and EM-oriented feature engineering baselines.

C. Ablation Study

In this section, we decompose the modules in the DSR framework and analyze their advantages in pairs.

DSR Mining Efficiency: Quick sort vs. Pruned sort. We compare our DSR mining algorithms: Prune-SE and Prune-IES, against Quick sort [71]. We record the runtime of these approaches as the number of *description* (donated by k hereafter) varies. We show our results in Figure 7.

We have the following observations:

(1) Pruned Sort performs better than Quick Sort. The Pruned Sort has a lower median runtime for all parameters. Furthermore, Pruned Sort has a relatively smaller interquartile range [72], which means it is more stable.

(2) For multiple-DSR in Prune-IES ($k \geq 2$), the speedup ratio increases with the growth of k. As the dashed line shows, the average time of Quick Sort has an upward trend as k increases, while Pruned Sort obviously slows down from $k = 3$ to $k = 4$. As shown in line 7 of Algorithm 4, each iteration of mining k-DSR involves further pruning based on the search space of $(k-1)$-DSR, thus leading to a better speedup.

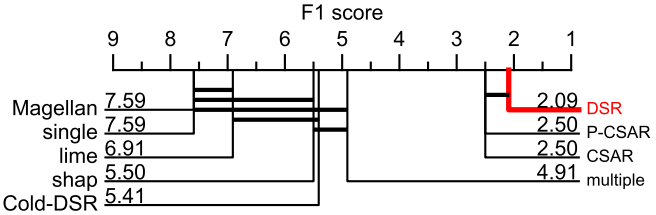


Fig. 6: Critical difference diagram for our DSR (bold red line) and other feature engineering baselines under the statistical level of 0.1.

Similarity Evaluation: P-CSAR vs. LIME vs. SHAP. We compare the efficiency of our similarity evaluation method P-CSAR with two widely used feature importance assessment methods LIME [28] and SHAP [29]. To ensure the same range of similarity scores as in the sampling strategy, we normalize the values generated from LIME and SHAP.

As the comparison result in Figure 6, we can see that P-CSAR significantly outperforms other methods.

Dimension Analysis of DSR: Single vs. Multiple. We analyze the performance of single- and multiple-DSR in feature engineering by splitting the mined DSR set according to k. We also compared DSR with Magellan [23], which uses data types and the single description `length` in the antecedents of the rules.

We observe that the Magellan and single-DSR have the same *F1 score ranking*, and both are significantly lower than multiple-DSR. Compared to Magellan, single-DSR has achieved similar performance by bringing in four kinds of corresponding *descriptions*. However, relying on a single description alone is still insufficient to recommend optimal

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

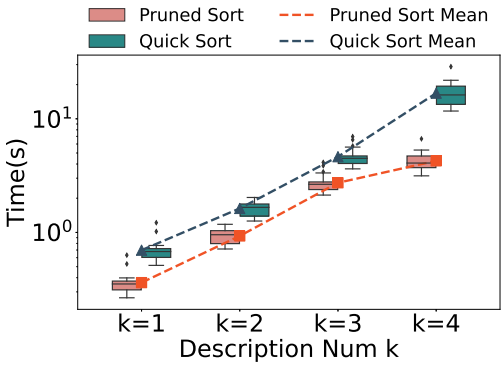


Fig. 7: Box plot of runtime for Quicksort and PrunedSE on all datasets. The dashed line shows the trend of average runtime(s) as the description num(k) grows. The mining algorithm is Prune-SE for $k = 1$ and Prune-ISE for $k > 1$. Our runtime is presented on a logarithmic scale.

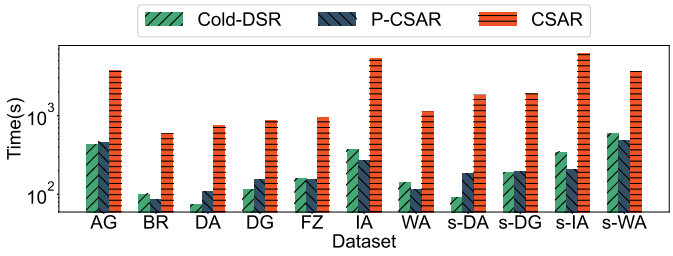


Fig. 8: Runtime of different sampling strategies on EM datasets. Our runtime is presented on a logarithmic scale.

similarities. Furthermore, by combining single- and multiple-DSR, we achieve the best F1 score ranking (the bold red line) among all methods. This suggests that with the incorporation of multiple-DSR, we can recommend a more comprehensive and effective combination of similarities.

Different Sampling Strategy. In this section, we compare the feature engineering performance of different sampling strategies that are based on multiple arm identification (MAI), presented as CSAR, P-CSAR, and Cold-DSR. In contrast to the other two methods, Cold-DSR adopts settled parameters and treats all datasets as historical data.

We present the critical difference diagram [73] of the methods in Figure 6. It reports the *mean ranking of F1 score* among all EM datasets. Competitors falling in one clique (the bold horizontal line) have no statistical difference. P-CSAR has higher *F1 score ranking* than Cold-DSR and has achieved such good approximation performance that there is *no statistically significant difference in F1 score* with CSAR. More importantly, P-CSAR reduces runtime by nearly 10 times as shown in Figure 8. It is obvious that P-CSAR and CSAR both outperform SHAP [28] and LIME [29], which are popular feature evaluation methods used in [74], [75]. Moreover, with runtime similar to P-CSAR, Cold-DSR outperforms Magellan and serves as a feasible cold-start solution.

Takeaway: P-CSAR is a more accurate similarity evaluator for DSR, compared to widely used ML model feature importance methods, SHAP and LIME. Multiple-DSR recommends better similarities than simple DSR, and the acceleration ratio of Stratified Sorting rises with the dimension of *descriptions*. Cold-DSR could be a better cold-start solution than Magellan.

D. Sensitivity Analysis

Fixing $\text{sup} = 0.33$, and $\theta = 0.83$ may not achieve the best performance for some EM datasets. Therefore, we conduct the sensitivity analysis on both parameters. We also empirically test a variant of DSR by searching these parameters with Bayesian Optimization, called Bayes-DSR.

A comparative analysis between sup **and** θ **on DSR mining.**

We fix one parameter as default and analyze the variations on the end-to-end runtime and rule numbers in mining algorithms as the other parameter rises. We applied the large-scale dataset WDC benchmark to get a better observation of the sensitivity of both parameters. Figure 9 shows our average results.

We have the following observations:

(1) sup exhibits better robustness compared to θ. We compute the variance in the running time given different θ, and it is 71 times that given different sup.

(2) The amount of DSR decreases rapidly as both parameters increase. For example, when θ changes from 0.76 to 0.78, the number of single-DSR is reduced to **20%** of the original. In terms of sup, the number of two-dimensional DSR has decreased by 3,300 when sup is set to 0.4 compared to the scenario with $\text{sup} = 0.32$.

(3) Only by selecting sup, θ from a specific range can we get a rational DSR set. For example, when $\text{sup} < 0.32$, more than $180k$ RuleNum is given by DSR, and the DSR degrades to AutoML-EM. Besides, when $\text{sup} > 0.48$, all multiple-DSR vanishes, and the DSR degrades to Magellan. As Example 5 shows, this may lead to missing certain key features.

Example 5. *Considering* ***Brew Factory Name*** *attribute in the Beer dataset, it contains numerous strings separated by spaces. From Definition 2, we recommend* `AWN` *to describe the entity pairs and select similarity functions.*

Suppose we change θ from 0.84 to 0.83, We observe an increased F1 score and an additional feature dimension in the Beer dataset. By analyzing the DSR set, we find a newly added DSR as follows:

$$(Length > 38.2) \wedge (AWN > 8.5) \rightarrow Levenshtein_{3gram} \quad (9)$$

The above rule recommends one more feature for ***Brew Factory Name****: Levenshtein-3gram, a sequence-based similarity function, which can increase the F1 score by 1.3%.*

From the above example, setting the parameters too high would limit DSR to recommend more similarities. To recommend appropriate parameters, we build Bayes-DSR, which uses priori Bayesian Optimization [76] to search parameters for DSR. According to the result in Figure 9, the search space is set as (0.32,0.48) and (0.76,0.84) for sup and θ separately.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

TABLE III: An end-to-end comparison between AutoML-EM, Bayes-DSR, and DSR. The performance of AutoML-EM and DSR are consistent with Table II.

Data Source	Dataset	AutoML-EM	Bayes-DSR	DSR
Magellan Repository	Fodors-Zagats(FZ)	100	**100**	**100**
	iTunes-Amazon(IA)	71.7	83	**100**
	s-iTunes-Amazon	96.4	98.36	**100**
	Beer(BR)	93.6	**94.4**	**94.4**
	DBLP-GoogleScholar(DG)	88.3	**90.65**	90.57
	DBLP-ACM(DA)	94.7	94.4	**94.96**
	s-DBLP-GoogleScholar	94.5	95.03	**95.22**
	s-DBLP-ACM	98.5	98.85	**98.98**
	Amazon-Google(AG)	56	56.79	**62.31**
	Walmart-Amazon(WA)	45.16	50.95	**51.16**
	s-Walmart-Amazon	75.74	**77.22**	76.4
WDC-xlarge	computers	71.54	74.44	**74.62**
	cameras	69	71.98	**72.94**
	shoes	57.35	61.6	**61.72**
	watches	83.25	84.53	**84.82**
	all-xlarge	71.91	**74.8**	74.79
WDC-large	computers	73.55	75.89	**76.59**
	cameras	67.34	71.14	**71.52**
	shoes	63.68	70.25	**70.4**
	watches	78.26	**80.78**	80.52
	all-large	70.33	**73.34**	73.18
WDC-medium	computers	72.19	**74.32**	73.18
	cameras	60.77	**65.98**	64.61
	shoes	62.24	68.14	**74.05**
	watches	69.79	**72.1**	67.55
	all-medium	67.15	68.69	**69.19**
WDC-small	computers	78.49	**79.7**	78.96
	cameras	72.13	**74.74**	74.03
	shoes	69	**74**	72.27
	watches	69.02	72.39	**75.37**
	all-small	70.47	**75.23**	74.49
All Above	average	74.58	77.39	**78.35**

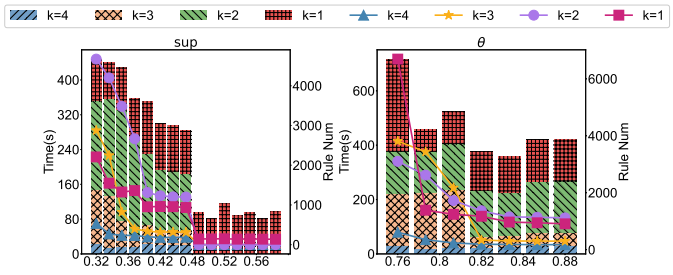


Fig. 9: Analysis of Parameter θ and sup of DSR Mining on WDC Benchmark. The bar chart shows the average runtime and the line plot shows the average quantity of mined DSR. k is the number of *descriptions* in DSR.

We use the F1 score as the objective function. In Table III, we show our comparison result.

On most datasets, Bayes-DSR outperforms AutoML-EM within a single iteration. Furthermore, we applied a t-test between Bayes-DSR and DSR. We get $p = 0.21$, which is higher than the threshold at the significance level $\alpha = 0.05$. Therefore, Bayes-DSR has no statistical difference with DSR, indicating that DSR demonstrates good robustness to parameters. Besides, it is recommended to set $\text{sup} = 0.33$ and $\theta = 0.81$ as a cold-start setting, where most datasets achieved satisfactory performance in our search process. Therefore, many datasets in the Magellan Repository such as Beer, structured-iTunes-Amazon, and DBLP-Scholar would only need to run DSR once to get satisfactory results.

VI. Related Work

Entity Matching is a long-standing challenge in database management, information retrieval, machine learning, and statistics. Since the 1970s, declarative matching rules have been extensively studied, both pre-defined and synthesized. In the 2000s, early Machine Learning Methods were introduced in EM, using statistics and textual similarities such as Jaccard and Edit Distance [21], [77]. In the early 2010s, Crowd-sourcing techniques [2], [78] took over EM by merging manual annotations on tuple pairs. In the mid-2010s, clustering methods and active learning implied a promising direction to enhance ML-based EM, which is still advancing in recent years [56], [79], [80]. Since 2017, the EM community has started to introduce deep learning models to EM solvers. Solutions based on Pre-trained Large Language Models(PLM) [1], [3], [11], [12] have been state-of-the-art. More recently, the research field of EM has witnessed a blossom of advanced machine learning methods, including Domain Adaptions [81], [82], Multi-task Representation Learning [32], Federated Learning [83], Entity Fairness [19], [84], etc. In addition, considerable research effort has also been put into the evaluation [19] and enhancement of EM model fairness [84], a timely topic in machine learning. The explainability of EM [17], [85]–[90] also attracts broad attention across the community. Some works in this direction tackle the explainability of PLM-based EM [87], [88]. Regarding RF-based EM, existing works focus on the explainability of machine learning models like feature importance [86] and counterfactual [17], [88]. As an extension of our previous work [9], DSR explains features of RF-based EM solutions with data-driven rules from training data, which has not been explored in the existing literature.

DSR is not limited to RF-based EM. For example, Large Language Model(LLM)-based EM [68], [91] require in-context examples to prompt LLMs but are limited by similarities. DSR could potentially recommend diverse similarity functions for LLM in-context example selection. Another interesting adaption for DSR is to replace RF with other rule-based EM solutions like GBF [92]. We refer interested readers to our technical report [45] for a full related work.

VII. Conclusion

In this paper, we propose the Description-Similarity Rule (DSR), a data-driven set of rules for feature engineering on Entity Matching models. We propose four descriptions to capture high-level data statistics for similarity computation. To reduce the online computation of model retraining, we design a workflow for DSR mining with multiple acceleration algorithms, novel theoretically guaranteed sampling, and a pruning heuristic algorithm. Our experiment shows that DSR outperforms existing feature engineering-based EM solutions, and achieves comparable performance with deep learning methods. For our next step, we plan to work on DSR antecedent extensions and design algorithms to derive approximate solutions for DSR mining.

Acknowledgements

This work is supported by National Natural Science Foundation of China (NSFC) (62232005, 62202126).

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Y. Li, J. Li, Y. Suhara, A. Doan, and W. Tan, "Deep entity matching with pre-trained language models," *Proc. VLDB Endow.*, vol. 14, no. 1, pp. 50–60, 2020.

[2] S. Das, P. S. G. C., A. Doan, J. F. Naughton, G. Krishnan, R. Deep, E. Arcaute, V. Raghavendra, and Y. Park, "Falcon: Scaling up hands-off crowdsourced entity matching to build cloud services," in *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD Conference 2017, Chicago, IL, USA, May 14-19, 2017* (S. Salihoglu, W. Zhou, R. Chirkova, J. Yang, and D. Suciu, eds.), pp. 1431–1446, ACM, 2017.

[3] R. Peeters and C. Bizer, "Dual-objective fine-tuning of BERT for entity matching," *Proc. VLDB Endow.*, vol. 14, no. 10, pp. 1913–1921, 2021.

[4] A. Doan, A. Y. Halevy, and Z. G. Ives, *Principles of Data Integration*. Morgan Kaufmann, 2012.

[5] X. Ding, H. Wang, G. Li, H. Li, Y. Li, and Y. Liu, "Iot data cleaning techniques: A survey," *Intell. Converged Networks*, vol. 3, no. 4, pp. 325–339, 2022.

[6] P. Wang, W. Zheng, J. Wang, and J. Pei, "Automating entity matching model development," in *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*, pp. 1296–1307, IEEE, 2021.

[7] J. Wang, G. Li, J. X. Yu, and J. Feng, "Entity matching: How similar is similar," *Proc. VLDB Endow.*, vol. 4, no. 10, pp. 622–633, 2011.

[8] F. Neutatz, F. Biessmann, and Z. Abedjan, "Enforcing constraints for machine learning systems via declarative feature selection: An experimental study," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021* (G. Li, Z. Li, S. Idreos, and D. Srivastava, eds.), pp. 1345–1358, ACM, 2021.

[9] H. Wang, X. Ding, J. Li, and H. Gao, "Rule-based entity resolution on database with hidden temporal information," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 11, pp. 2199–2212, 2018.

[10] Z. Chen, Q. Chen, F. Fan, Y. Wang, Z. Wang, Y. Nafa, Z. Li, H. Liu, and W. Pan, "Enabling quality control for entity resolution: A human and machine cooperation framework," in *34th IEEE International Conference on Data Engineering, ICDE 2018, Paris, France, April 16-19, 2018*, pp. 1156–1167, IEEE Computer Society, 2018.

[11] U. Brunner and K. Stockinger, "Entity matching with transformer architectures - A step forward in data integration," in *Proceedings of the 23rd International Conference on Extending Database Technology, EDBT 2020, Copenhagen, Denmark, March 30 - April 02, 2020* (A. Bonifati, Y. Zhou, M. A. V. Salles, A. Böhm, D. Olteanu, G. H. L. Fletcher, A. Khan, and B. Yang, eds.), pp. 463–473, OpenProceedings.org, 2020.

[12] M. Ebraheem, S. Thirumuruganathan, S. R. Joty, M. Ouzzani, and N. Tang, "Distributed representations of tuples for entity resolution," *Proc. VLDB Endow.*, vol. 11, no. 11, pp. 1454–1467, 2018.

[13] R. Wang, Y. Li, and J. Wang, "Sudowoodo: Contrastive self-supervised learning for multi-purpose data integration and preparation," in *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3-7, 2023*, pp. 1502–1515, IEEE, 2023.

[14] Z. Miao, Y. Li, and X. Wang, "Rotom: A meta-learned data augmentation framework for entity matching, data cleaning, text classification, and beyond," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021* (G. Li, Z. Li, S. Idreos, and D. Srivastava, eds.), pp. 1303–1316, ACM, 2021.

[15] M. Stonebraker and I. F. Ilyas, "Data integration: The current status and the way forward," *IEEE Data Eng. Bull.*, vol. 41, pp. 3–9, 2018.

[16] N. Barlaug, "Tailoring entity matching for industrial settings," in *CIKM '20: The 29th ACM International Conference on Information and Knowledge Management, Virtual Event, Ireland, October 19-23, 2020* (M. d'Aquin, S. Dietze, C. Hauff, E. Curry, and P. Cudré-Mauroux, eds.), pp. 3217–3220, ACM, 2020.

[17] J. Wang and Y. Li, "Minun: evaluating counterfactual explanations for entity matching," in *DEEM '22: Proceedings of the Sixth Workshop on Data Management for End-To-End Machine Learning Philadelphia, PA, USA, 12 June 2022* (M. Boehm, P. Varma, and D. Xin, eds.), pp. 7:1–7:11, ACM, 2022.

[18] P. Li, X. Cheng, X. Chu, Y. He, and S. Chaudhuri, "Auto-fuzzyjoin: Auto-program fuzzy similarity joins without labeled examples," in *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021* (G. Li, Z. Li, S. Idreos, and D. Srivastava, eds.), pp. 1064–1076, ACM, 2021.

[19] N. Shahbazi, N. Danevski, F. Nargesian, A. Asudeh, and D. Srivastava, "Through the fairness lens: Experimental analysis and evaluation of entity matching," *Proc. VLDB Endow.*, vol. 16, p. 3279–3292, July 2023.

[20] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[21] S. Chaudhuri, B. Chen, V. Ganti, and R. Kaushik, "Example-driven design of efficient record matching queries," in *Proceedings of the 33rd International Conference on Very Large Data Bases, University of Vienna, Austria, September 23-27, 2007* (C. Koch, J. Gehrke, M. N. Garofalakis, D. Srivastava, K. Aberer, A. Deshpande, D. Florescu, C. Y. Chan, V. Ganti, C. Kanne, W. Klas, and E. J. Neuhold, eds.), pp. 327–338, ACM, 2007.

[22] K. Kim and C. L. Giles, "Financial entity record linkage with random forests," in *Proceedings of the Second International Workshop on Data Science for Macro-Modeling*, DSMM'16, (New York, NY, USA), Association for Computing Machinery, 2016.

[23] P. Konda, S. Das, P. S. G. C., A. Doan, A. Ardalan, J. R. Ballard, H. Li, F. Panahi, H. Zhang, J. F. Naughton, S. Prasad, G. Krishnan, R. Deep, and V. Raghavendra, "Magellan: Toward building entity matching management systems," *Proc. VLDB Endow.*, vol. 9, no. 12, pp. 1197–1208, 2016.

[24] M. Feurer, K. Eggensperger, S. Falkner, M. Lindauer, and F. Hutter, "Auto-sklearn 2.0: The next generation," *CoRR*, vol. abs/2007.04074, 2020.

[25] D. W. Aha and R. L. Bankert, "A comparative evaluation of sequential feature selection algorithms," in *Learning from Data - Fifth International Workshop on Artificial Intelligence and Statistics, AISTATS 1995, Key West, Florida, USA, January 4-7, 1995. Proceedings* (D. Fisher and H. Lenz, eds.), pp. 199–206, Springer, 1995.

[26] W. Zuo, D. Zhang, and K. Wang, "On kernel difference-weighted k-nearest neighbor classification," *Pattern Analysis and Applications*, vol. 11, pp. 247–257, 08 2007.

[27] F. Nie, H. Huang, X. Cai, and C. H. Q. Ding, "Efficient and robust feature selection via joint 2, 1-norms minimization," in *Advances in Neural Information Processing Systems 23: 24th Annual Conference on Neural Information Processing Systems 2010. Proceedings of a meeting held 6-9 December 2010, Vancouver, British Columbia, Canada* (J. D. Lafferty, C. K. I. Williams, J. Shawe-Taylor, R. S. Zemel, and A. Culotta, eds.), pp. 1813–1821, Curran Associates, Inc., 2010.

[28] M. T. Ribeiro, S. Singh, and C. Guestrin, ""why should I trust you?": Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016* (B. Krishnapuram, M. Shah, A. J. Smola, C. C. Aggarwal, D. Shen, and R. Rastogi, eds.), pp. 1135–1144, ACM, 2016.

[29] B. Karlas, D. Dao, M. Interlandi, B. Li, S. Schelter, W. Wu, and C. Zhang, "Data debugging with shapley importance over end-to-end machine learning pipelines," *CoRR*, vol. abs/2204.11131, 2022.

[30] X. Li, Q. Guo, G. Zhang, S. Ye, G. He, Y. Yao, R. Zhang, Y. Hao, Z. Du, and W. Zheng, "Fasttuning: Enabling fast and efficient hyper-parameter tuning with partitioning and parallelism of search space," *IEEE Trans. Parallel Distributed Syst.*, vol. 35, no. 7, pp. 1174–1188, 2024.

[31] A. Bogatu, N. W. Paton, M. Douthwaite, S. Davie, and A. Freitas, "Cost-effective variational active entity resolution," in *37th IEEE International Conference on Data Engineering, ICDE 2021, Chania, Greece, April 19-22, 2021*, pp. 1272–1283, IEEE, 2021.

[32] J. Tu, J. Fan, N. Tang, P. Wang, G. Li, X. Du, X. Jia, and S. Gao, "Unicorn: A unified multi-tasking model for supporting matching tasks in data integration," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 84:1–84:26, 2023.

[33] J. Wang, G. Li, and J. Feng, "Extending string similarity join to tolerant fuzzy token matching," *ACM Trans. Database Syst.*, vol. 39, Jan. 2014.

[34] S. Sarawagi and A. Kirpal, "Scaling up the ALIAS duplicate elimination system," in *Proceedings of the 19th International Conference on Data Engineering, March 5-8, 2003, Bangalore, India* (U. Dayal, K. Ramamritham, and T. M. Vijayaraman, eds.), pp. 783–785, IEEE Computer Society, 2003.

[35] S. Chaudhuri, B. Chen, V. Ganti, and R. Kaushik, "Example-driven design of efficient record matching queries," in *Proceedings of the 33rd International Conference on Very Large Data Bases, University of Vienna, Austria, September 23-27, 2007* (C. Koch, J. Gehrke, M. N. Garofalakis, D. Srivastava, K. Aberer, A. Deshpande, D. Florescu, C. Y. Chan, V. Ganti, C. Kanne, W. Klas, and E. J. Neuhold, eds.), pp. 327–338, ACM, 2007.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

[36] M. Jaro, "Advances in record-linkage methodology as applied to matching the 1985 census of tampa, florida," *JASA: Journal of the American Statistical Association*, 1989.

[37] M. K. Vijaymeena and K. Kavitha, "A survey on similarity measures in text mining," *Machine Learning & Applications An International Journal*, vol. 3, no. 1, pp. 19–28, 2016.

[38] R. R. Shamir, Y. Duchin, J. Kim, G. Sapiro, and N. Harel, "Continuous dice coefficient: a method for evaluating probabilistic segmentations," *CoRR*, vol. abs/1906.11031, 2019.

[39] S. Ontañón, "An overview of distance and similarity functions for structured data," *Artif. Intell. Rev.*, vol. 53, no. 7, pp. 5309–5351, 2020.

[40] H. He and J. D. Choi, "The stem cell hypothesis: Dilemma behind multi-task learning with transformer encoders," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021* (M. Moens, X. Huang, L. Specia, and S. W. Yih, eds.), pp. 5555–5577, Association for Computational Linguistics, 2021.

[41] D. Tian, M. Li, Y. Shen, and S. Han, "Intelligent mining of safety hazard information from construction documents using semantic similarity and information entropy," *Eng. Appl. Artif. Intell.*, vol. 119, p. 105742, 2023.

[42] S. Helmer, "Measuring the structural similarity of semistructured documents using entropy," in *Proceedings of the 33rd International Conference on Very Large Data Bases, University of Vienna, Austria, September 23-27, 2007* (C. Koch, J. Gehrke, M. N. Garofalakis, D. Srivastava, K. Aberer, A. Deshpande, D. Florescu, C. Y. Chan, V. Ganti, C. Kanne, W. Klas, and E. J. Neuhold, eds.), pp. 1022–1032, ACM, 2007.

[43] D. C. Brown, "A cryptanalysis of the autokey cipher using the index of coincidence," in *Proceedings of the ACMSE 2018 Conference, Richmond, KY, USA, March 29-31, 2018* (K. Wong, C. Shen, and D. Brown, eds.), pp. 5:1–5:8, ACM, 2018.

[44] S. P. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–136, 1982.

[45] Y. Tang, Z. Liang, H. Wang, X. Ding, T. Mu, and H. Hu, "Technical report on "description-similarity rules: Towards flexible feature engineering for entity matching"," 2024. Accessed: 2024-12-05.

[46] J. Peng, B. Ding, J. Wang, K. Zeng, and J. Zhou, "One size does not fit all: A bandit-based sampler combination framework with theoretical guarantees," in *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022* (Z. G. Ives, A. Bonifati, and A. E. Abbadi, eds.), pp. 531–544, ACM, 2022.

[47] S. Bubeck, T. Wang, and N. Viswanathan, "Multiple identifications in multi-armed bandits," in *Proceedings of the 30th International Conference on Machine Learning, ICML 2013, Atlanta, GA, USA, 16-21 June 2013*, vol. 28 of *JMLR Workshop and Conference Proceedings*, pp. 258–265, JMLR.org, 2013.

[48] P. Kolpaczki, V. Bengs, and E. Hüllermeier, "Identifying top-k players in cooperative games via shapley bandits," in *Proceedings of the LWDA 2021 Workshops: FGWM, KDML, FGWI-BIA, and FGIR, Online, September 1-3, 2021* (T. Seidl, M. Fromm, and S. Obermeier, eds.), vol. 2993 of *CEUR Workshop Proceedings*, pp. 133–144, CEUR-WS.org, 2021.

[49] C. Finn and S. Levine, "Meta-learning: from few-shot learning to rapid reinforcement learning," *ICML 2019 Meta-Learning Tutorial*, 2019.

[50] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017* (D. Precup and Y. W. Teh, eds.), vol. 70 of *Proceedings of Machine Learning Research*, pp. 1126–1135, PMLR, 2017.

[51] T. Mu, H. Wang, S. Zheng, Z. Liang, C. Wang, X. Shao, and Z. Liang, "Tsc-automl: Meta-learning for automatic time series classification algorithm selection," in *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3-7, 2023*, pp. 1032–1044, IEEE, 2023.

[52] M. Mohri, A. Rostamizadeh, and A. Talwalkar, *Foundations of Machine Learning*. Adaptive computation and machine learning, MIT Press, 2012.

[53] Y. Sun, Z. Zheng, S. Song, and F. Chiang, "Confidence bounded replica currency estimation," in *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022* (Z. G. Ives, A. Bonifati, and A. E. Abbadi, eds.), pp. 730–743, ACM, 2022.

[54] X. Ding, Y. Lu, H. Wang, C. Wang, Y. Liu, and J. Wang, "Dafdiscover: Robust mining algorithm for dynamic approximate functional dependencies on dirty data," *Proc. VLDB Endow.*, vol. 17, no. 11, pp. 3484–3496, 2024.

[55] X. Ding, Y. Liu, H. Wang, C. Wang, Y. Song, D. Yang, and J. Wang, "Efficient relaxed functional dependency discovery with minimal set cover," in *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*, pp. 3519–3531, IEEE, 2024.

[56] X. Ding, H. Wang, J. Su, M. Wang, J. Li, and H. Gao, "Leveraging currency for repairing inconsistent and incomplete data," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 3, pp. 1288–1302, 2022.

[57] X. Ding, Y. Song, H. Wang, D. Yang, C. Wang, and J. Wang, "Clean4tsdb: A data cleaning tool for time series databases," *Proc. VLDB Endow.*, vol. 17, no. 12, pp. 4377–4380, 2024.

[58] S. Aggarwal, S. Mitra, and A. Bhattacharya, "Skycover: Finding range-constrained approximate skylines with bounded quality guarantees," in *21st International Conference on Management of Data, COMAD 2016, Pune, India, March 11-13, 2016* (A. Deshpande, B. Ravindran, and S. Ranu, eds.), pp. 1–12, Computer Society of India, 2016.

[59] A. Primpeli, R. Peeters, and C. Bizer, "The wdc training dataset and gold standard for large-scale product matching," in *Companion Proceedings of The 2019 World Wide Web Conference*, WWW '19, (New York, NY, USA), p. 381–386, Association for Computing Machinery, 2019.

[60] D. Paulsen, Y. Govind, and A. Doan, "Sparkly: A simple yet surprisingly strong tf/idf blocker for entity matching," *Proc. VLDB Endow.*, vol. 16, p. 1507–1519, Feb. 2023.

[61] S. Mudgal, H. Li, T. Rekatsinas, A. Doan, Y. Park, G. Krishnan, R. Deep, E. Arcaute, and V. Raghavendra, "Deep learning for entity matching: A design space exploration," in *Proceedings of the 2018 International Conference on Management of Data, SIGMOD Conference 2018, Houston, TX, USA, June 10-15, 2018* (G. Das, C. M. Jermaine, and P. A. Bernstein, eds.), pp. 19–34, ACM, 2018.

[62] U. Brunner and K. Stockinger, "Entity matching with transformer architectures - A step forward in data integration," in *Proceedings of the 23rd International Conference on Extending Database Technology, EDBT 2020, Copenhagen, Denmark, March 30 - April 02, 2020* (A. Bonifati, Y. Zhou, M. A. V. Salles, A. Böhm, D. Olteanu, G. H. L. Fletcher, A. Khan, and B. Yang, eds.), pp. 463–473, OpenProceedings.org, 2020.

[63] Y. Sun, J. Zhu, X. Xu, X. Xu, Y. Sun, S. Song, X. Li, and X. Yuan, "Win-win: On simultaneous clustering and imputing over incomplete data," *Proc. VLDB Endow.*, vol. 17, no. 11, pp. 3045–3057, 2024.

[64] R. Tibshirani, "Regression shrinkage and selection via the lasso: a retrospective," *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 73, no. 3, pp. 267–288, 2011.

[65] K. Zhu, X. Feng, X. Du, Y. Gu, W. Yu, H. Wang, Q. Chen, Z. Chu, J. Chen, and B. Qin, "An information bottleneck perspective for effective noise filtering on retrieval-augmented generation," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024* (L. Ku, A. Martins, and V. Srikumar, eds.), pp. 1044–1069, Association for Computational Linguistics, 2024.

[66] Z. Ma, J. Du, and S. Zhou, "Noise-robust training with dynamic loss and contrastive learning for distantly-supervised named entity recognition," in *Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 9-14, 2023* (A. Rogers, J. L. Boyd-Graber, and N. Okazaki, eds.), pp. 10119–10128, Association for Computational Linguistics, 2023.

[67] S. Si, Z. Cai, S. Zeng, G. Feng, J. Lin, and B. Chang, "SANTA: separate strategies for inaccurate and incomplete annotation noise in distantly-supervised named entity recognition," in *Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, July 9-14, 2023* (A. Rogers, J. L. Boyd-Graber, and N. Okazaki, eds.), pp. 3883–3896, Association for Computational Linguistics, 2023.

[68] M. Fan, X. Han, J. Fan, C. Chai, N. Tang, G. Li, and X. Du, "Cost-effective in-context learning for entity resolution: A design space exploration," in *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*, pp. 3696–3709, IEEE, 2024.

[69] S. Wu, Q. Wu, H. Dong, W. Hua, and X. Zhou, "Blocker and matcher can mutually benefit: A co-learning framework for low-resource entity resolution," *Proc. VLDB Endow.*, vol. 17, no. 3, pp. 292–304, 2023.

[70] T. Deng, W. Fan, P. Lu, X. Luo, X. Zhu, and W. An, "Deep and collective entity resolution in parallel," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*, pp. 2060–2072, IEEE, 2022.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.

[71] C. A. R. Hoare, "Algorithm 64: Quicksort," *Commun. ACM*, vol. 4, no. 7, p. 321, 1961.

[72] G. U. Yule and M. G. Kendall, "An introduction to the theory of statistics," *The Economic Journal*, vol. 60, no. 240, 1968.

[73] J. Demsar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.

[74] Q. Li, Y. Ji, M. Zhu, X. Zhu, and L. Sun, "Unsupervised feature selection using chronological fitting with shapley additive explanation (SHAP) for industrial time-series anomaly detection," *Appl. Soft Comput.*, vol. 155, p. 111426, 2024.

[75] J. Verhaeghe, M. J. V. D. Donckt, F. Ongenae, and S. V. Hoecke, "Powershap: A power-full shapley feature selection method," in *Machine Learning and Knowledge Discovery in Databases - European Conference, ECML PKDD 2022, Grenoble, France, September 19-23, 2022, Proceedings, Part I* (M. Amini, S. Canu, A. Fischer, T. Guns, P. K. Novak, and G. Tsoumakas, eds.), vol. 13713 of *Lecture Notes in Computer Science*, pp. 71–87, Springer, 2022.

[76] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States* (P. L. Bartlett, F. C. N. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, eds.), pp. 2960–2968, 2012.

[77] O. Benjelloun, H. Garcia-Molina, D. Menestrina, Q. Su, S. E. Whang, and J. Widom, "Swoosh: a generic approach to entity resolution," *VLDB J.*, vol. 18, no. 1, pp. 255–276, 2009.

[78] C. Chai, G. Li, J. Li, D. Deng, and J. Feng, "A partial-order-based framework for cost-effective crowdsourced entity resolution," *VLDB J.*, vol. 27, no. 6, pp. 745–770, 2018.

[79] R. Wu, S. Chaba, S. Sawlani, X. Chu, and S. Thirumuruganathan, "Zeroer: Entity resolution using zero labeled examples," in *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020* (D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, eds.), pp. 1149–1164, ACM, 2020.

[80] V. V. Meduri, L. Popa, P. Sen, and M. Sarwat, "A comprehensive benchmark framework for active learning methods in entity matching," in *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020* (D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, eds.), pp. 1133–1147, ACM, 2020.

[81] J. Tu, J. Fan, N. Tang, P. Wang, C. Chai, G. Li, R. Fan, and X. Du, "Domain adaptation for deep entity resolution," in *SIGMOD '22: International Conference on Management of Data, Philadelphia, PA, USA, June 12 - 17, 2022* (Z. G. Ives, A. Bonifati, and A. E. Abbadi, eds.), pp. 443–457, ACM, 2022.

[82] J. Tu, X. Han, J. Fan, N. Tang, C. Chai, G. Li, and X. Du, "DADER: hands-off entity resolution with domain adaptation," *Proc. VLDB Endow.*, vol. 15, no. 12, pp. 3666–3669, 2022.

[83] X. Qin, C. Chai, N. Tang, J. Li, Y. Luo, G. Li, and Y. Zhu, "Synthesizing privacy preserving entity resolution datasets," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*, pp. 2359–2371, IEEE, 2022.

[84] N. Shahbazi, J. Wang, Z. Miao, and N. Bhutani, "Fairness-aware data preparation for entity matching," in *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*, pp. 3476–3489, IEEE, 2024.

[85] Z. Chen, Q. Chen, B. Hou, Z. Li, and G. Li, "Towards interpretable and learnable risk analysis for entity resolution," in *Proceedings of the 2020 International Conference on Management of Data, SIGMOD Conference 2020, online conference [Portland, OR, USA], June 14-19, 2020* (D. Maier, R. Pottinger, A. Doan, W. Tan, A. Alawini, and H. Q. Ngo, eds.), pp. 1165–1180, ACM, 2020.

[86] A. Ebaid, S. Thirumuruganathan, W. G. Aref, A. K. Elmagarmid, and M. Ouzzani, "EXPLAINER: entity resolution explanations," in *35th IEEE International Conference on Data Engineering, ICDE 2019, Macao, China, April 8-11, 2019*, pp. 2000–2003, IEEE, 2019.

[87] S. Thirumuruganathan, M. Ouzzani, and N. Tang, "Explaining entity resolution predictions: Where are we and what needs to be done?," in *Proceedings of the Workshop on Human-In-the-Loop Data Analytics, HILDA@SIGMOD 2019, Amsterdam, The Netherlands, July 5, 2019*, pp. 10:1–10:6, ACM, 2019.

[88] T. Teofili, D. Firmani, N. Koudas, V. Martello, P. Merialdo, and D. Srivastava, "Effective explanations for entity resolution models," in *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*, pp. 2709–2721, IEEE, 2022.

[89] N. Barlaug, "LEMON: explainable entity matching," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 8, pp. 8171–8184, 2023.

[90] A. Baraldi, F. D. Buono, F. Guerra, M. Paganelli, and M. Vincini, "An intrinsically interpretable entity matching system," in *Proceedings 26th International Conference on Extending Database Technology, EDBT 2023, Ioannina, Greece, March 28-31, 2023* (J. Stoyanovich, J. Teubner, N. Mamoulis, E. Pitoura, J. Mühlig, K. Hose, S. S. Bhowmick, and M. Lissandrini, eds.), pp. 645–657, OpenProceedings.org, 2023.

[91] H. Li, S. Li, F. Hao, C. J. Zhang, Y. Song, and L. Chen, "Booster: Leveraging large language models for enhancing entity resolution," in *Companion Proceedings of the ACM on Web Conference 2024, WWW 2024, Singapore, Singapore, May 13-17, 2024* (T. Chua, C. Ngo, R. K. Lee, R. Kumar, and H. W. Lauw, eds.), pp. 1043–1046, ACM, 2024.

[92] R. Singh, V. V. Meduri, A. K. Elmagarmid, S. Madden, P. Papotti, J. Quiané-Ruiz, A. Solar-Lezama, and N. Tang, "Synthesizing entity matching rules by examples," *Proc. VLDB Endow.*, vol. 11, no. 2, pp. 189–202, 2017.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:45:22 UTC from IEEE Xplore. Restrictions apply.